



FOR JAVA DEVELOPERS MOVING TO NESTJS

Spring Accepted This?

Spring Boot to NestJS, episode 6: request validation

Validation is three jobs: convert types, handle unknown fields, check rules.

Spring and NestJS put request strictness in different places. In our app Spring checks bad values with Bean Validation, but Jackson converts types and drops unknown fields.

NestJS checks nothing at the boundary until you bind a ValidationPipe. The NestJS side below has it switched on, which is the point of the comparison, not a default.

Versions on this sheet were current on 29 September 2026: NestJS 12, Spring Boot 4.1.1 with Jackson 3.

In scope

- Type conversion, unknown fields and rule checking, as three separate jobs
- Why Spring accepted a string amount and an extra field
- What transform, whitelist and forbidNonWhitelisted each do
- What reaches the handler without whitelist
- The same rules in Bean Validation and class-validator

Not in scope

- Turning the rejection into a response: that is the next episode, exception filters

The generic Spring 400 body shown is this app's default error body, not a Spring rule.

01 The measurement

STRING AMOUNT "10000"	Spring 201, converted to 10000 / NestJS 400
EXTRA "isAdmin": true	Spring 201, field dropped / NestJS 400, property isAdmin should not exist
EMPTY BODY, JUNK VALUES	both 400
JACKSON, COERCION OFF	Cannot coerce String value ("10000") to long
JACKSON, FAIL_ON_UNKNOWN_PROPERTIES	Unrecognized property "isAdmin"
NESTJS, TRANSFORM, NO WHITELIST	handler received amountInMinorUnits, currency, isAdmin, role
NESTJS, WHITELIST ON	handler received amountInMinorUnits, currency

Spring at its defaults; NestJS with ValidationPipe on.



02 The NestJS pipe

transform turns plain JSON into your class. whitelist removes undeclared properties.
forbidNonWhitelisted rejects the request instead of removing them.

```
nestjs-api/src/main.ts
```

```
app.useGlobalPipes(  
  new ValidationPipe({  
    whitelist: true,  
    forbidNonWhitelisted: true,  
    transform: true,  
  }),  
);
```

"Start by binding ValidationPipe at the application level"

NestJS documentation, Validation, checked 2026-09-29.

03 The same rules, two languages

The concepts transfer; the syntax does not map perfectly. Spring checks the currency with a pattern, NestJS with an allowed list, and the integer check is a separate decorator.

CreatePaymentRequest.java

```
public record CreatePaymentRequest(
    @NotNull @Min(value = 1, message = "amountInMinorUnits must be at least 1")
    Long amountInMinorUnits,
    @NotNull @Pattern(regexp = "USD|EUR|GBP|ZAR", message = "currency must be a supported ISO
code")
    String currency,
    @NotBlank(message = "idempotencyKey is required") String idempotencyKey) {}
```

create-payment.dto.ts

```
export class CreatePaymentRequest {
    @IsInt({ message: 'amountInMinorUnits must be an integer number of minor units' })
    @Min(1, { message: 'amountInMinorUnits must be at least 1' })
    amountInMinorUnits!: number;

    @IsIn(['USD', 'EUR', 'GBP', 'ZAR'], { message: 'currency must be a supported ISO code' })
    currency!: string;

    @IsString()
    @IsNotEmpty({ message: 'idempotencyKey is required' })
    idempotencyKey!: string;
}
```

04 Making Spring strict

Both behaviours are Jackson defaults and both can be switched off. In Spring Boot, deserialization features are set with `spring.jackson.deserialization.<feature_name>`. At the time of recording, Spring Boot 4.1.1 ships Jackson 3 and the imports moved to `tools.jackson.databind`.

JacksonLenienceTest.java

```
JsonMapper strict = JsonMapper.builder()
    .enable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
    .build();
```

with `FAIL_ON_UNKNOWN_PROPERTIES`: Unrecognized property "isAdmin"

Run on a real machine, 2026-09-29, by `scripts/verify-validation.sh` in the course repository.



Before you ship a NestJS endpoint

1. Bind ValidationPipe globally
2. Turn on whitelist, and forbidNonWhitelisted if extra fields should fail
3. Turn on transform so decorators run on a real instance
4. Put an integer check where Java had a Long
5. In Spring, decide deliberately on FAIL_ON_UNKNOWN_PROPERTIES
6. Know which layer owns conversion, shape and rules

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-validation.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS: Validation https://docs.nestjs.com/techniques/validation

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-29 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.